



9주차: 실용적 대형언어모델 구축 RAG&LoRA

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

arxiv.org

<https://arxiv.org/pdf/2005.11401>

Introduction

문제 제시

대규모 언어모델이 파라미터 안에 많은 지식을 저장하고 있지만, 필요한 정보를 정확히 꺼내거나 최신 정보로 바꾸는 데 한계가 있다.

해결방안 제시

모델 내부 지식과 외부 문서 저장소를 함께 사용하는 RAG 구조를 제안

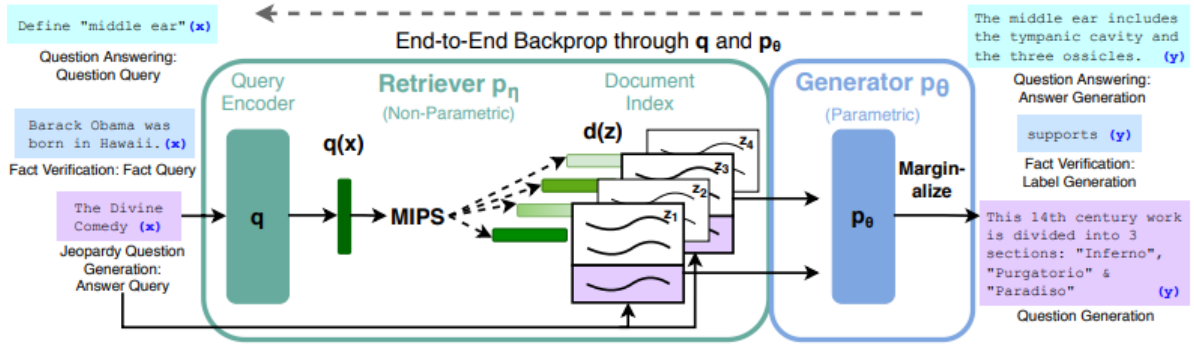
1. 입력이 들어오면 검색기(DPR)가 위키피디아에서 관련 문서 찾음
2. 생성기(BART)는 입력과 검색된 문서를 함께 참고해 답을 생성함
3. 이때, 논문은 하나의 답변 전체에서 같은 문서를 참고 & 각 토큰마다 다른 문서를 참고하는 두가지 구조를 비교한다.

실험 결과

⇒ RAG는 여러 지식 집약적 NLP 작업에서 기존 모델보다 더 좋은 성능을 보였고, 생성 품질 면에서도 더 사실적이고 구체적이며 다양한 응답을 만들어냈다.

⇒ 외부 메모리를 교체할 수 있기에 모델 전체를 다시 학습하지 않고도 최신 지식을 반영할 수 있다.

Methods



RAG 모델은 입력 시퀀스 x 를 사용하여 텍스트 문서 z 를 검색하고 추가 컨텍스트로 이를 사용하여 타겟 y 를 생성한다.

1. Retriever $p_{\eta}(z|x)$: 파라미터가 η 이며, 주어진 쿼리 x 에 대하여 텍스트 구절에 대한 분포를 반환한다.
(각 문서가 얼마나 관련 있는 지를 확률 분포 형태로 주는 것임 = 입력 x 가 주어졌을 때, 문서 z 가 선택될 확률)
2. Generator $p_{\theta}(y_i|x, z, y_{1:i-1})$: 이전 토큰들 $y_{1:i-1}$, 원래 입력 x , 검색된 구절 z 의 컨텍스트를 기반으로 현재 토큰 y_i 를 생성한다.

Retriever와 Generator를 end-to-end로 학습시키기 위해 검색된 문서를 latent variable로 취급한다. 저자들은 생성된 텍스트에 대한 분포를 생성하기 위해 latent 문서를 다른 방식으로 marginalize하는 두 가지 모델을 제안하였다.
(특정 문서 하나만 딱 고정하지 않고 가능한 여러 문서들을 다 고려해서 전체 확률 계산한 것이다.)

1. **RAG-Sequence**: 동일한 문서를 사용하여 각 대상 토큰을 예측한다.
2. **RAG-Token**: 다른 문서를 기반으로 각 대상 토큰을 예측한다.

2.1 Models

RAG-Sequence

“검색된 상위 K 개 문서 각각에 대해 “이 문서만 참고해서 전체 답변을 생성할 확률”을 계산한 뒤, 그 문서가 선택될 확률로 가중합해서 최종 답변 확률을 구하는 방식”

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

RAG-Sequence 모델은 검색된 동일한 문서를 사용하여 전체 시퀀스를 생성한다. 검색된 문서를 하나의 latent variable로 취급하여 top-K 근사를 통해 seq2seq 확률 $p(y|x)$ 를 얻는다.

⇒ 상위 K개의 문서는 retriever를 사용하여 검색되고 generator는 각 문서에 대한 출력 시퀀스 확률을 생성한 다음 이를 marginalize한다.

RAG-Token

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

RAG-Token 모델은 각 대상 토큰에 대해 다른 latent 문서를 사용하여 그에 따라 marginalize할 수 있다. 이를 통해 generator는 답변을 생성할 때 여러 문서에서 콘텐츠를 선택할 수 있다.

⇒ 상위K개의 문서는 retriever를 사용하여 검색되고, generator는 marginalize하기 전에 각 문서에 대한 다음 출력 토큰에 대한 분포를 생성하고, 이 프로세스를 그 다음 출력 토큰에 대하여 반복한다.

2.2 Retriever: DPR

$$p_{\eta}(z|x) \propto \exp(\mathbf{d}(z)^{\top} \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{BERT}_d(z), \quad \mathbf{q}(x) = \text{BERT}_q(x)$$

$$\mathbf{d}(z) = \text{BERT}_d(z)$$

document encoder에서 생성된 dense한 표현

$$\mathbf{q}(x) = \text{BERT}_q(x)$$

query encoder에서 생성된 query 표현

두 인코더 모두 BERT-base 모델을 기반으로 한다.

가장 높은 사전 확률 $p_{\eta}(z|x)$ 를 갖는 k개의 문서 z인 $\text{top-}k(p_{\eta}(\cdot|x))$ 를 계산하는 것은 Maximum Inner Product Search (MIPS) 문제이며, 이는 대략적으로 sub-linear time 내에 풀 수 있다. DPR의 사전 학습된 bi-encoder를 사용하여 retriever를 초기화하고 문서 인덱스를 구축한다. 이 retriever는 TriviaQA 질문과 Natural Questions에 대한 답변이 포함된 문서를 검색하도록 학습되었다. 문서 인덱스를 **non-parametric memory**라고 한다.

2.3 Generator: BART

Generator $p_{\theta}(y_i|x, z, y_{1:i-1})$ 는 임의의 인코더-디코더를 사용하여 모델링할 수 있다. 본 논문에서는 사전 학습된 seq2seq transformer인 BART-large를 사용하였다. BART에서 생성할 때 입력 x와 검색된 콘텐츠 z는 간단히 concat되어 결합된다. BART generator의 파라미터 θ 를 **parametric memory**라고 한다.

2.4 Training

어떤 문서를 검색해야 하는지에 대한 직접적인 supervision 없이 retriever와 generator를 공동으로 학습시킨다. 입력/출력 쌍 (x_j, y_j) 로 구성된 fine-tuning 데이터가 주어지면 Adam을 사용한 stochastic gradient descent을 사용하여 각 타겟의 negative marginal log-likelihood

$\sum_j -\log p(y_j|x_j)$ 를 최소화한다. 학습 중에 document encoder $BERT_d$ 를 업데이트하는 것은 문서 인덱스를 주기적으로 업데이트해야 하므로 비용이 많이 든다. 저자들은 이 단계가 강력한 성능을 위해 필요하지 않다고 생각하고 document encoder와 문서 인덱스를 고정한 채 query encoder $BERT_q$ 와 BART generator만 fine-tuning한다.

2.5 Decoding

테스트 시에는 RAG-Sequence와 RAG-Token이 서로 다른 방법으로 $\operatorname{argmax}_y p(y|x)$ 를 근사한다.

RAG-Token

RAG-Token 모델은 transition probability가 다음과 같은 표준적인 autoregressive seq2seq generator로 볼 수 있다.

$$p'_{\theta}(y_i|x, y_{1:i-1}) = \sum_{z \in \operatorname{top-k}(p(\cdot|x))} p_{\eta}(z_i|x) p_{\theta}(y_i|x, z_i, y_{1:i-1})$$

디코딩하려면 $p'_{\theta}(y_i|x, y_{1:i-1})$ 을 표준 beam search 디코더에 넣으면 된다.

RAG-Sequence

RAG-Sequence의 경우, likelihood $p(y|x)$ 는 per-token likelihood로 분리되지 않으므로 한 번의 beam search로 해결할 수 없다. 대신 각 문서 z 에 대해 beam search를 실행하여 $p_{\theta}(y_i|x, z, y_{1:i-1})$ 을 사용하여 각 가설을 채점한다. 그러면 가설들의 집합 Y 가 생성되는데, 이 중 일부는 모든 문서의 beam에 나타나지 않았을 수 있다. 가설 y 의 확률을 추정하기 위해 beam에 y 가 나타나지 않는 각 문서 z 에 대해 추가 forward pass를 실행하고, generator 확률을 $p_{\eta}(z|x)$ 로 곱한 다음 marginal들에 대한 beam 전체의 확률을 합산한다. 이 디코딩 절차를 **Thorough Decoding**이라고 한다.

더 긴 출력 시퀀스의 경우 Y 의 크기가 커져 여러 forward pass가 필요할 수 있다. 보다 효율적인 디코딩을 위해 y 가 beam search에서 생성되지 않은 경우 $p_{\theta}(y|x, z_i) \approx 0$ 로 추가 근사를 할 수 있다. 이렇게 하면 Y 가 생성된 후 추가 forward pass를 실행할 필요가 없다. 이 디코딩 절차를 **Fast Decoding**이라고 한다.

Experiments

Open-domain Question Answering

Generation and Classification

	Model	NQ	TQA	WQ	CT
Closed Book	T5-11B [52]	34.5	- /50.1	37.4	-
	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open Book	REALM [20]	40.4	- / -	40.7	46.8
	DPR [26]	41.5	57.9 / -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

Model	Jeopardy		MSMARCO		FVR3	FVR2
	B-1	QB-1	R-L	B-1	Label	Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	<u>89.5</u>
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

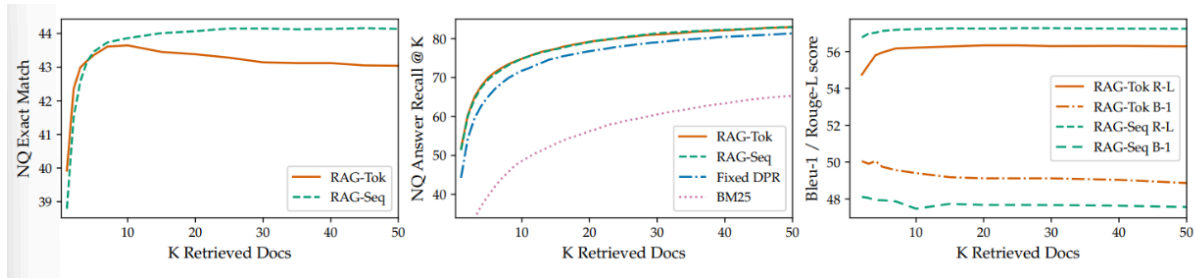
Additional Results

	MSMARCO	Jeopardy QGen
Gold	89.6%	90.0%
BART	70.7%	32.4%
RAG-Token	77.8%	46.8%
RAG-Seq.	83.5%	53.8%

Model	NQ	TQA	WQ	CT	Jeopardy-QGen		MSMarco		FVR-3	FVR-2
		Exact Match			B-1	QB-1	R-L	B-1	Label Accuracy	
RAG-Token-BM25	29.7	41.5	32.1	33.1	17.5	22.3	55.5	48.4		
RAG-Sequence-BM25	31.8	44.1	36.6	33.8	11.1	19.5	56.5	46.9	75.1	91.6
RAG-Token-Frozen	37.8	50.1	37.1	51.1	16.7	21.7	55.9	49.4	72.9	89.4
RAG-Sequence-Frozen	41.2	52.1	41.8	52.6	11.8	19.6	56.7	47.3		
RAG-Token	43.5	54.8	46.5	51.9	17.9	22.6	56.2	49.4	74.5	90.6
RAG-Sequence	44.0	55.8	44.9	53.4	15.3	21.5	57.2	47.5		

전체 tri-gram에 대한 고유 tri-gram의 비율을
비교한 표 ⇒ RAG-Sequence의 생성이
RAG-Token의 생성보다 더 다양하며, 둘 다 다
양성을 촉진하는 디코딩이 필요 없이 BART보
다 훨씬 더 다양하다는 것을 보여준다.

ablation 결과



검색된 문서 수에 따른 Natural Questions (NQ) 성능을 비교한 그래프

LoRA: Low-Rank Adaptation of Large Language Models

arxiv.org

<https://arxiv.org/pdf/2106.09685>

Introduction

문제 제시

대규모 언어모델을 새로운 작업에 맞게 적응시킬 때, 모든 파라미터를 다시 학습하는 전체 파인튜닝이 너무 비싸고 비효율적이다.

해결방안 제시

저자들은 사전 학습된 모델의 원래 가중치는 고정하고 대신 각 Transformer 층에 작은 저랭크 행렬을 추가해 그 부분만 학습하는 방식인 LoRA 방식을 제안한다.

“모델을 새로운 작업에 맞게 조정할 때 필요한 가중치 변화는 사실 전체 차원을 다 쓸 필요가 없고, 훨씬 낮은 랭크의 변화로도 충분할 수 있다”

장점

- 학습해야 할 파라미터 수가 매우 적어서 메모리와 저장 비용이 크게 줄어든다.
- 전체 파인튜닝과 비슷하거나 더 좋은 성능을 내면서도 학습이 더 빠르다.
- adapter 처럼 네트워크 구조를 더 깊게 만들지 않기 때문에 추론 시 추가 지연이 없다.

▼ Terminologies

- d_{model} : Transformer 레이어의 입력 및 출력 차원 크기
- W_q, W_k, W_v, W_o : self-attention 모듈에서 query, key, value, output projection 행렬
- W 또는 W_0 : 사전 학습된 가중치 행렬
- ΔW : Adaptation 중에 누적된 기울기 업데이트
- r : LoRA 모듈의 rank

Problem Statement

Φ 로 parameterize된 사전 학습된 autoregressive model $p_\Phi(y|x)$ 가 주어졌다고 가정하자.

예를 들어 $P_\Phi(y|x)$ 는 Transformer 아키텍처를 기반으로 하는 GPT와 같은 일반적인 multi-task learner일 수 있다.

요약, 기계 독해(MRC), SQL에 대한 자연어(NL2SQL)와 같은 다운스트림 조건부 텍스트 생성 task에 사전 학습된 이 모델을 적용하는 것을 고려하자. 각 다운스트림 task는 context-target 쌍의 학습 데이터셋으로 표시된다.

$$Z = \{(x_i, y_i)\}_{i=1, \dots, N}$$

여기서 x_i 와 y_i 는 모두 토큰 시퀀스이다.

전체 fine-tuning 중에 모델은 사전 학습된 가중치 Φ_0 로 초기화되고, 조건부 언어 모델링 목적 함수를 최대화하기 위해 반복적으로 기울기를 따라 $\Phi_0 + \Delta\Phi$ 로 업데이트된다.

$$\max_{\Phi} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y < t))$$

전체 fine-tuning의 주요 단점 중 하나는 각 다운스트림 task에 대해 차원 $|\Delta\Phi|$ 과 $|\Phi_0|$ 가 같다는 것이다. 따라서 사전 학습된 모델이 큰 경우 fine-tuning된 모델의 많은 독립적인 인스턴스를 저장하고 배포하는 것은 가능하더라도 어려울 수 있다.

본 논문에서는 task별 $\Delta\Phi = \Delta\Phi(\Theta)$ 가 $|\Theta| \ll |\Phi_0|$ 인 파라미터 Θ 로 인코딩되는 더 파라미터 효율적인 접근 방식을 채택한다. 따라서 $\Delta\Phi$ 를 찾는 작업은 Θ 에 대해 최적화된다.

$$\max_{\Theta} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log(P_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y < t))$$

본 논문은 low-rank 표현을 사용하여 계산 및 메모리 효율적인 $\Delta\Phi$ 를 인코딩할 것을 제안한다. 사전 학습된 모델이 GPT-3 175B인 경우 학습 가능한 파라미터의 수 $|\Theta|$ 는 $|\Phi_0|$ 의 0.01% 정도로 작을 수 있다.

Aren't Existing Solutions Good Enough?

본 논문이 해결하려는 문제는 결코 새로운 것이 아니다. Transfer learning이 시작된 이래 수십 개의 연구들이 모델 adaptation을 보다 파라미터 및 계산 효율적으로 만들기 위해 노력했다. 예를 들어 언어 모델링을 사용하면 효율적인 adaptation과 관련하여 두 가지 주요 전략이 있다.

1. Adapter layer 추가
2. 입력 레이어 activation의 일부 형식 최적화

But, 두 전략 모두 특히 대규모 모델이나 latency에 민감한 시나리오에서 한계가 있다.

Adapter layer들은 inference latency를 도입한다.

다양한 adapter 변형이 있다.

- Transformer 블록당 두 개의 adapter layer가 있는 원래 디자인
- 블록당 하나만 있지만 추가 LayerNorm이 있는 최신 디자인

레이어를 정리하거나 멀티태스킹 설정을 활용하여 전체 latency를 줄일 수 있지만 adapter layer의 추가 컴퓨팅을 우회하는 직접적인 방법은 없다. 이는 adapter layer가 추가할 수 있는 FLOP를 제한하는 작은 bottleneck 차원을 가짐으로써 소수의 파라미터(원래 모델의 1% 미만)를 갖도록 설계되었기 때문에 문제가 아닌 것처럼 보인다.

그러나 대규모 신경망은 latency를 낮게 유지하기 위해 하드웨어 병렬 처리에 의존하며 adapter layer는 순차적으로 처리되어야 한다.

이는 배치 크기가 일반적으로 1만큼 작은 온라인 inference 설정에서 차이를 만든다. 단일

GPU의 GPT-2에서 inference를 실행하는 것과 같이 모델 병렬 처리가 없는 일반적인 시나리오에서는 bottleneck 차원이 매우 작은 경우에도 adapter를 사용할 때 latency가 눈에 띄게 증가하는 것을 볼 수 있다.

Batch Size	32	16	1
Sequence Length	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4±0.8	338.0±0.6	19.8±2.7
Adapter ^L	1482.0±1.0 (+2.2%)	354.8±0.5 (+5.0%)	23.9±2.1 (+20.7%)
Adapter ^H	1492.2±1.0 (+3.0%)	366.3±0.5 (+8.4%)	25.8±2.2 (+30.3%)

Adapter 파라미터를 여러 번 중복 저장하지 않는 한 추가 깊이에는 더 많은 synchronous GPU 연산이 필요하기 때문에 모델을 샤딩(sharding)해야 할 때 이 문제는 더욱 악화된다.

프롬프트를 직접 최적화하는 것은 어렵다.

Prefix tuning은 최적화하기 어렵고 그 성능이 학습 가능한 파라미터에서 단조적이지 않게 변한다. 더 근본적으로, adaptation을 위해 시퀀스 길이의 일부를 사용하면 다운스트림 task를 처리하는 데 사용할 수 있는 시퀀스 길이가 필연적으로 줄어들어 다른 방법에 비해 프롬프트 튜닝 성능이 떨어진다.

Method

Low-Rank-Parametrized Update Matrices

신경망에는 행렬 곱셈을 수행하는 많은 레이어가 포함되어 있다. 이러한 레이어의 가중치 행렬은 일반적으로 full-rank를 갖는다. 특정 task에 적응할 때, 사전 학습된 언어 모델은 더 작은 subspace로의 랜덤 projection에도 불구하고 여전히 효율적으로 학습할 수 있으며, 낮은 **내재적 차원 (intrinsic dimension)**을 가진다.

이에 영감을 받아 가중치에 대한 업데이트도 adaptation 중에 낮은 **intrinsic rank**를 갖는다고 가정한다. 사전 학습된 가중치 행렬 $W_0 \in \mathbb{R}^{d \times k}$ 의 경우,

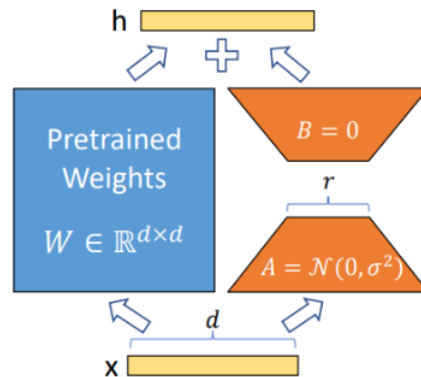
$$W_0 + \Delta W = W^0 + BA$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$

로 표현하여 업데이트를 제한한다.

학습하는 동안 W_0 는 고정되고 기울기 업데이트를 받지 않는 반면 A와 B는 학습 가능한 파라미터를 포함한다. W_0 와 $\Delta W = BA$ 는 모두 동일한 입력으로 곱해지고 각각의 출력 벡터는 좌표 방향으로 합산된다. $h = W_0 x$ 인 경우 수정된 forward pass는 다음과 같다.

$$h = w_0x + \Delta x = W_0x + BAx$$



위 그림에서 reparameterization을 설명한다.

초기화 방식

- A는 랜덤 가우시안 분포로 초기화
- B는 0으로 초기화

⇒ $BA=0$ 이 되어 LoRA가 추가되더라도 초기에는 원래 pretrained 모델의 동작을 그대로 유지할 수 있다.

업데이트 크기 조절

∇W_x 를 α/r 로 스케일링한다. (여기서 α 는 r 의 상수)

즉, rank r 이 바뀌더라도 업데이트 크기가 지나치게 커지거나 작아지지 않도록 조절

여기서 α 의 역할은 업데이트 강조를 조절하는 것이다.

전체 fine-tuning의 일반화

보다 일반적인 형태의 fine-tuning을 통해 사전 학습된 파라미터의 부분집합을 학습할 수 있다.

LoRA는 한 단계 더 나아가 adaptation 중에 full-rank를 갖기 위해 가중치 행렬에 대한 누적 기울기 업데이트가 필요하지 않다.

⇒ 모든 가중치 행렬에 LoRA를 적용하고 모든 바이어스를 학습할 때 LoRA rank r 을 사전 학습된 가중치 행렬의 rank로 설정하여 전체 fine-tuning의 표현력을 대략적으로 복구한다.

⇒ 학습 가능한 파라미터의 수가 증가함에 따라 LoRA 학습은 대략적으로 원래 모델 학습으로 수렴되는 반면 어댑터 기반 방법은 MLP로, prefix 기반 방법은 긴 입력 시퀀스를 취할 수 없는 모델로 수렴된다.

추가적인 inference latency 없음

프로덕션 환경에 배포할 때, $W = W_0 + BA$ 를 명시적으로 계산 및 저장하고 평소와 같이 inference를 수행할 수 있다. W_0 과 BA 는 모두 $R^{d \times k}$ 이다. 다른 다운스트림 task로 전환해야 하는 경우 BA 를 뺀 다음 다른 B_0A_0 을 추가하여 W_0 을 복구할 수 있으며, 메모리 오버헤드가 거의 없는 빠른 연산이다.

⇒ 결정적으로 이것은 fine-tuning된 모델과 비교하여 inference 중에 추가 latency를 도입하지 않도록 보장한다.

Applying LoRA to Transformer

학습 가능한 파라미터의 수를 줄이기 위해 신경망에서 가중치 행렬의 모든 부분집합에 LoRA를 적용할 수 있다. Transformer 아키텍처에는 self-attention 모듈에 4개의 가중치 매트릭스 W_q, W_k, W_v, W_o 가 있고 MLP 모듈에는 2개가 있다. W_q (또는 W_k, W_v)를 차원 $d_{model} \times d_{model}$ 의 단일 행렬로 취급한다. 출력 차원은 일반적으로 attention head로 분할된다. 저자들은 단순성과 파라미터 효율성을 위해 다운스트림 task에 대한 attention 가중치만 튜닝하고 MLP 모듈을 고정하는 것으로 연구를 제한하였다.

실질적인 이점

1. 메모리와 스토리지 사용량 감소이다.

Adam으로 학습된 대형 Transformer의 경우 고정 파라미터에 대한 optimizer 상태를 저장할 필요가 없으므로 $r \ll d_{model}$ 인 경우 VRAM 사용량을 최대 2/3까지 줄인다.

GPT-3 175B의 경우 학습 중 VRAM 소비를 1.2TB에서 350GB로 줄인다. $r=4$ 이고 query와 key projection 행렬만 튜닝되면 체크포인트 크기가 약 10,000배 (350GB에서 35MB로) 감소한다. 이를 통해 훨씬 적은 수의 GPU로 학습하고 입출력 병목 현상을 피할 수 있다.

2. 모든 파라미터가 아닌 LoRA 가중치만 교환함으로써 훨씬 저렴한 비용으로 배포 중에 task 사이를 전환할 수 있다는 것이다.

이를 통해 사전 학습된 가중치를 VRAM에 저장하는 시스템에서 즉석에서 교환할 수 있는 많은 맞춤형 모델을 생성할 수 있다. 또한 대부분의 파라미터에 대한 기울기를 계산할 필요가 없기 때문에 전체 fine-tuning에 비해 GPT-3 175B에서 학습하는 동안 25% 속도 향상을 보인다.

한계점

예를 들어 추가 inference latency를 제거하기 위해 A와 B를 W로 흡수하기로 선택한 경우 단일 forward pass에서 A와 B가 다른 여러 task에 대한 입력을 일괄 처리하는 것은 간단하지 않다. 가중치를 병합하지 않고 latency가 중요하지 않은 시나리오의 경우 배치에서 샘플에 사용할 LoRA 모듈을 동적으로 선택할 수 있다.

Empirical Experiments

1. Baseline

- **FT**: Fine-Tuning
- FT^{Top2} : 마지막 두 레이어만 튜닝
- **BitFit**
- $Adap^H$: 오리지널 adapter tuning
- $Adap^L$: MLP 모듈 뒤와 LayerNorm 뒤에만 adapter layer 적용
- $Adap^P$: AdapterFusion
- $Adap^D$: AdapterDrop (몇몇 adapter layer를 drop)

2. Result

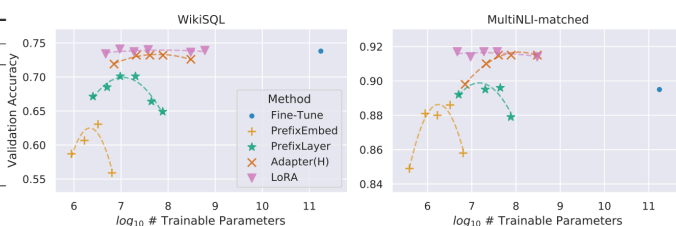
Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoBbase (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoBbase (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoBbase (Adp ^L)*	0.3M	87.1 $\pm$ 0.1	94.2 $\pm$ 1.1	88.5 $\pm$ 1.1	60.8 $\pm$ 4.1	93.1 $\pm$ 1.1	90.2 $\pm$ 0.9	71.5 $\pm$ 2.7	89.7 $\pm$ 3.3	84.4
RoBbase (Adp ^P)*	0.9M	87.3 $\pm$ 1.1	94.7 $\pm$ 3.3	88.4 $\pm$ 1.1	62.6 $\pm$ 9.9	93.0 $\pm$ 2.2	90.6 $\pm$ 0.9	75.9 $\pm$ 2.2	90.3 $\pm$ 1.1	85.4
RoBbase (LoRA)	0.3M	87.5 $\pm$ 3.3	95.1 $\pm$ 2.2	89.7 $\pm$ 7.7	63.4 $\pm$ 1.2	93.3 $\pm$ 3.3	90.8 $\pm$ 1.1	86.6 $\pm$ 7.7	91.5 $\pm$ 2.2	87.2
RoBlarge (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoBlarge (LoRA)	0.8M	90.6 $\pm$ 2.2	96.2 $\pm$ 5.5	90.9 $\pm$ 1.2	68.2 $\pm$ 1.9	94.9 $\pm$ 3.3	91.6 $\pm$ 1.1	87.4 $\pm$ 2.5	92.6 $\pm$ 2.2	89.0
RoBlarge (Adp ^L)†	3.0M	90.2 $\pm$ 3.3	96.1 $\pm$ 3.3	90.2 $\pm$ 7.7	68.3 $\pm$ 1.0	94.8 $\pm$ 3.3	91.9 $\pm$ 1.1	83.8 $\pm$ 2.9	92.1 $\pm$ 7.7	88.4
RoBlarge (Adp ^P)†	0.8M	90.5 $\pm$ 3.3	96.6 $\pm$ 2.2	89.7 $\pm$ 12.2	67.8 $\pm$ 2.5	94.8 $\pm$ 3.3	91.7 $\pm$ 3.3	80.1 $\pm$ 2.9	91.9 $\pm$ 4.4	87.9
RoBlarge (Adp ^H)†	6.0M	89.9 $\pm$ 5.5	96.2 $\pm$ 3.3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 3.3	92.1 $\pm$ 1.1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoBlarge (Adp ^H)†	0.8M	90.3 $\pm$ 3.3	96.3 $\pm$ 5.5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 3.3	91.5 $\pm$ 1.1	72.9 $\pm$ 2.9	91.5 $\pm$ 5.5	86.4
RoBlarge (LoRA)†	0.8M	90.6 $\pm$ 2.2	96.2 $\pm$ 5.5	90.2 $\pm$ 1.0	68.2 $\pm$ 1.9	94.8 $\pm$ 3.3	91.6 $\pm$ 2.2	85.2 $\pm$ 1.1	92.3 $\pm$ 5.5	88.6
DeBXXL (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeBXXL (LoRA)	4.7M	91.9 $\pm$ 2.2	96.9 $\pm$ 2.2	92.6 $\pm$ 6.6	72.4 $\pm$ 1.1	96.0 $\pm$ 1.1	92.9 $\pm$ 1.1	94.9 $\pm$ 4.4	93.0 $\pm$ 2.2	91.3

다양한 adaptation 방법을 적용한 RoBERTabase, RoBERTalarge, DeBERTaXXL에 대한 GLUE 벤치마크

Model & Method	# Trainable Parameters	BLEU	NIST	E2E NLG Challenge MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)*	11.09M	67.3 $\pm$ 6.6	8.50 $\pm$ 0.7	46.0 $\pm$ 2.2	70.7 $\pm$ 2.2	2.44 $\pm$ 0.1
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4 $\pm$ 1.1	8.85 $\pm$ 0.2	46.8 $\pm$ 2.2	71.8 $\pm$ 1.1	2.53 $\pm$ 0.2
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)*	0.88M	69.1 $\pm$ 1.1	8.68 $\pm$ 0.3	46.3 $\pm$ 0.0	71.4 $\pm$ 2.2	2.49 $\pm$ 0.0
GPT-2 L (Adapter ^L)*	23.00M	68.9 $\pm$ 3.3	8.70 $\pm$ 0.4	46.1 $\pm$ 1.1	71.3 $\pm$ 2.2	2.45 $\pm$ 0.2
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4 $\pm$ 1.1	8.89 $\pm$ 0.2	46.8 $\pm$ 2.2	72.0 $\pm$ 2.2	2.47 $\pm$ 0.2

E2E NLG Challenge에서 다양한 adaptation 방법을 적용한 GPT-2 medium (M)과 GPT-2 large (L)에 대한 비교 결과

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1



다양한 adaptation 방법을 적용한 GPT-3 175B의 성능 비교 결과

Understanding the Low-Rank Updates

Low-rank 구조는 여러 실험을 병렬로 실행할 수 있도록 하드웨어 진입 장벽을 낮출 뿐만 아니라 업데이트 가중치가 사전 학습된 가중치와 어떻게 상관되는지에 대한 더 나은 해석성을

제공한다. 저자들은 GPT-3 175B에 대한 연구에 집중하여 task 수행에 부정적인 영향을 미치지 않으면서 학습 가능한 파라미터를 가장 많이 줄였다.

저자들은 다음 질문에 답하기 위해 일련의 경험적 연구를 수행하였다.

1. 파라미터 예산에 대한 제약 조건이 주어지면 다운스트림 성능을 최대화하기 위해 사전 학습된 Transformer에서 가중치 행렬의 어떤 부분집합을 튜닝해야 하는가?
2. 최적의 adaptation 행렬 ΔW 가 정말 rank가 부족한가? 그렇다면 실제로 사용하기 좋은 rank는 무엇인가?
3. ΔW 와 W 사이의 관계는 무엇인가? ΔW 는 W 와 높은 상관관계가 있는가? ΔW 는 W 에 비해 얼마나 큰가?

1. Transformer의 어떤 가중치 행렬에 LoRA를 적용해야 하는가?

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

GPT-3의 가중치 행렬 종류에 대한 정확도

2. LoRA를 위한 최적의 rank r 은 무엇인가?

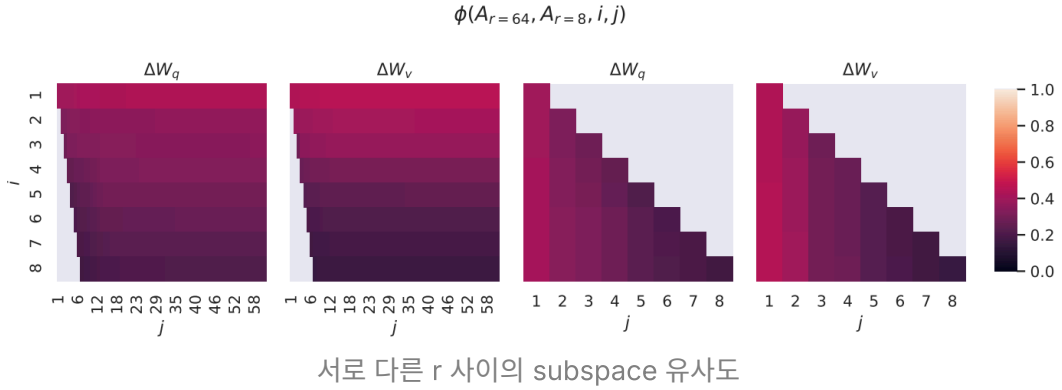
	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL ($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

다양한 rank r 에 대한 정확도

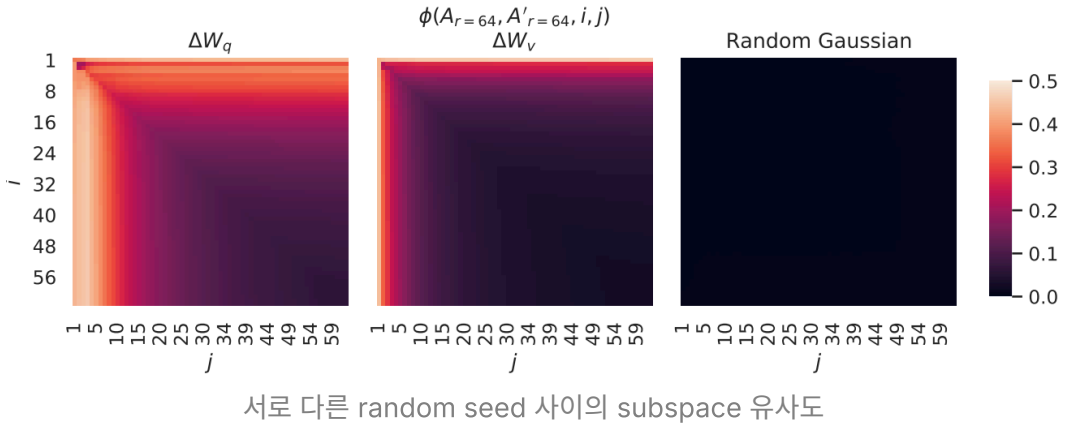
저자들은 Grassmann distance를 기반으로 다음과 같이 두 subspace의 유사도를 측정하였다.

$$\phi(A_1, A_2, i, j) = \frac{\|U_{A_1}^{i\top} U_{A_2}^j\|_F^2}{\min(i, j)} \in [0, 1]$$

여기서 U_A^i 는 top i singular vector에 해당하는 UA의 열이다. $\phi(\cdot)$ 가 1이면 subspace들이 완전히 겹친다는 것을 의미하며, 0이면 완전한 분리를 의미한다.



위의 singular vector에 해당하는 방향은 $A_r = 8$ 과 $A_r = 64$ 사이에서 상당히 겹치지만 다른 방향은 그렇지 않다. 특히, $A_r=8$ 와 $A_r=64$ 의 ΔW_v 와 ΔW_v 는 유사도가 0.5보다 크게 차원 1의 subspace를 공유하므로 GPT-3에 대한 다운스트림 task에서 $r=10$ 이 매우 잘 수행되는 이유를 설명할 수 있다.



$\Rightarrow \Delta W_q$ 는 ΔW_v 보다 intrinsic rank가 더 높은 것으로 보인다.

3. Adaptation 행렬 ΔW 는 W 와 어떻게 비교되는가?

$U^\top W V^\top$ 를 계산하여 W 를 ΔW 의 r 차원 subspace에 project한다. 여기서 U 와 V 는 ΔW 의 왼쪽과 오른쪽 singular vector 행렬이다. 그런 다음 $\|U^\top W V^\top\|_F$ 와 $\|W\|$ 사이의 Frobenius norm을 비교한다. 비교를 위해 U , V 를 W 의 top r singular vector 또는 랜덤 행렬로 대체하여 $\|U^\top W V^\top\|_F$ 도 계산한다.

	$r = 4$			$r = 64$		
	ΔW_q	W_q	Random	ΔW_q	W_q	Random
$\ U^\top W_q V^\top\ _F =$	0.32	21.67	0.02	1.90	37.71	0.33
$\ W_q\ _F = 61.95$	$\ \Delta W_q\ _F = 6.91$			$\ \Delta W_q\ _F = 3.57$		

위 결과에서 다음과 같은 결론을 내릴 수 있다.

1. ΔW 는 랜덤 행렬에 비해 W 와 더 강한 상관관계를 가지며, 이는 ΔW 가 이미 W 에 있는 일부 feature를 증폭함을 나타낸다.
2. ΔW 는 W 의 top singular 방향을 반복하는 대신 강조되지 않은 방향만 증폭한다.
3. 증폭 계수는 다소 크다. $r=4$ 인 경우 $21.5 \approx 6.91/0.32$ 이다.